SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – High (Coding Challenge)

Course Code:	CSA0501	**Course Title:**	**Programming in Java**
CO Assessed:	CO1 – Precision (BL3,BL4)	**Assessment:**	Assessment Tool 1 – Coding Challenge
Weightage:		**Total Marks:**	100 (5 Questions × 20 Marks)
CO1 Statement:	*Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*		
Student Name:	ANUSREE A	**Reg. No.:**	1911260033
Section / Batch:	B / I	**Date:**	22.07.2008

Code of Conduct

I _ANUSREE A / 1911260033_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- *This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.*
- *No negative marking. Unanswered questions carry zero marks.*
- *No calculators or electronic devices permitted.*
- *Duration: 90 minutes.*

ANNEXURE

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

PROGRAM:

```
import java.util.Scanner;

class Book {

    private int bookId;

    private String title;

    private String author;

    private double price;

    private boolean issued;

    // Constructor

    Book(int id, String t, String a, double p) {

        bookId = id;

        title = t;

        author = a;

        price = p;

        issued = false;

    }

    // Getters
```

```
    int getBookId() {
        return bookId;
    }
    // Display Book
    void display() {
        System.out.println("Book ID: " + bookId);
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
                System.out.println("Status:  "  +  (issued  ?  "Issued"  :
"Available"));
    }
    // Issue Book
    void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued");
        } else {
            System.out.println("Book Already Issued");
        }
    }
    // Return Book
    void returnBook() {
        issued = false;
```

```
            System.out.println("Book Returned");
        }
    }
public class LibraryManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book books[] = new Book[10];
        int count = 0;
        while (true) {
            System.out.println("\n1.Add Book");
            System.out.println("2.Display Books");
            System.out.println("3.Search Book");
            System.out.println("4.Issue Book");
            System.out.println("5.Return Book");
            System.out.println("6.Exit");
            System.out.print("Enter Choice: ");
            int ch = sc.nextInt();
            if (ch == 1) {
                System.out.print("Enter Book ID: ");
                int id = sc.nextInt();
                sc.nextLine();
                System.out.print("Enter Title: ");
                String title = sc.nextLine();
```

```
            System.out.print("Enter Author: ");
            String author = sc.nextLine();
            System.out.print("Enter Price: ");
            double price = sc.nextDouble();
            books[count] = new Book(id, title, author, price);
            count++;
            System.out.println("Book Added");
        } else if (ch == 2) {
            for (int i = 0; i < count; i++)
                books[i].display();
        } else if (ch == 3) {
            System.out.print("Enter Book ID: ");
            int id = sc.nextInt();
            for (int i = 0; i < count; i++) {
                if (books[i].getBookId() == id)
                    books[i].display();
            }
        } else if (ch == 4) {
            System.out.print("Enter Book ID: ");
            int id = sc.nextInt();
            for (int i = 0; i < count; i++) {
                if (books[i].getBookId() == id)
                    books[i].issueBook();
```

```
                }
            } else if (ch == 5) {
                System.out.print("Enter Book ID: ");
                int id = sc.nextInt();
                for (int i = 0; i < count; i++) {
                    if (books[i].getBookId() == id)
                        books[i].returnBook();
                }
            } else if (ch == 6) {
                break;
            } else {
                System.out.println("Invalid Choice");
            }
        }
        sc.close();
    }
}
```

OUTPUT:

```
1.Add Book
2.Display Books
3.Search Book
4.Issue Book
5.Return Book
6.Exit
```

```
Enter Choice: 1

Enter Book ID: 101

Enter Title: JAVA PROGRAMMING

Enter Author: JAMES

Enter Price: 500

Book Added
```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

PROGRAM:

```
 import java.util.Scanner;

// Base Class

class Employee {

    int id;

String name;

    Employee(int id, String name) {

        this.id = id;

        this.name = name;

    }

    void calculateSalary() {

        System.out.println("Salary Calculation");

    }
```

```
    void display() {
        System.out.println("Employee ID : " + id);
        System.out.println("Employee Name : " + name);
    }
}
// Derived Class - Permanent Employee
class PermanentEmployee extends Employee {
    double basicSalary, bonus;
      PermanentEmployee(int id, String name, double basicSalary,
double bonus) {
        super(id, name);
        this.basicSalary = basicSalary;
        this.bonus = bonus;
    }
    @Override
    void calculateSalary() {
        double salary = basicSalary + bonus;
        display();
             System.out.println("Permanent Employee Salary = " +
salary);
    }
}
// Derived Class - Contract Employee
```

```
class ContractEmployee extends Employee {

    int hours;

    double rate;

     ContractEmployee(int id, String name, int hours, double rate)
{

        super(id, name);

        this.hours = hours;

        this.rate = rate;

    }

    @Override

    void calculateSalary() {

        double salary = hours * rate;

        display();

       System.out.println("Contract Employee Salary = " + salary);

    }

}

// Main Class

public class PayrollSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("1. Permanent Employee");

        System.out.println("2. Contract Employee");

        System.out.print("Enter Choice: ");
```

```
int choice = sc.nextInt();
Employee emp;
if (choice == 1) {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Basic Salary: ");
    double basic = sc.nextDouble();
    System.out.print("Enter Bonus: ");
    double bonus = sc.nextDouble();
    emp = new PermanentEmployee(id, name, basic, bonus);
} else {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Hours Worked: ");
    int hours = sc.nextInt();
```

```
            System.out.print("Enter Rate per Hour: ");
            double rate = sc.nextDouble();
            emp = new ContractEmployee(id, name, hours, rate);
        }
        emp.calculateSalary();   // Runtime Polymorphism
        sc.close();
    }
}
```

OUTPUT:

```
1. Permanent Employee
2. Contract Employee
Enter Choice: 1
Enter ID: 101
Enter Name: ANU
Enter Basic Salary: 50000.0
Enter Bonus: 500.0
Employee ID : 101
Employee Name : ANU
Permanent Employee Salary = 50500.0
```

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

PROGRAM:

```
import java.util.Scanner;

class Vehicle {

    void calculateRent(int days) {

    }

}

class Car extends Vehicle {

    void calculateRent(int days) {

        System.out.println("Car Rent = " + (days * 1000));

    }

}

class Bike extends Vehicle {

    void calculateRent(int days) {

        System.out.println("Bike Rent = " + (days * 500));

    }

}

class Bus extends Vehicle {

    void calculateRent(int days) {

        System.out.println("Bus Rent = " + (days * 2000));

    }
```

```
}
public class VehicleRental {
    public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
        Vehicle v;
 System.out.println("1.Car");
        System.out.println("2.Bike");
        System.out.println("3.Bus");
        System.out.print("Enter Choice: ");
        int ch = sc.nextInt();

        System.out.print("Enter Days: ");
        int days = sc.nextInt();

        if (ch == 1)
            v = new Car();
        else if (ch == 2)
            v = new Bike();
        else
            v = new Bus();
 v.calculateRent(days);   // Runtime Polymorphism
sc.close();
    }
```

```
}
```

OUTPUT:

```
1.Car

2.Bike

3.Bus

Enter Choice: 1

Enter Days: 5

Car Rent = 5000
```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

PROGRAM:

```
 import java.util.Scanner;

class BankAccount {

    private double balance;

 BankAccount(double balance) {

        this.balance = balance;

    }

void deposit(double amount) {

        balance += amount;

    }

 void withdraw(double amount) {
```

```
        if (amount <= balance)
            balance -= amount;
        else
            System.out.println("Insufficient Balance");
    }
void showBalance() {
        System.out.println("Balance = " + balance);
    }
}
public class BankDemo {
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter Initial Balance: ");
        double bal = sc.nextDouble();
 BankAccount b = new BankAccount(bal);
System.out.println("1.Deposit");
        System.out.println("2.Withdraw");
        System.out.println("3.Show Balance");
        System.out.print("Enter Choice: ");
        int ch = sc.nextInt();
  if (ch == 1) {
            System.out.print("Enter Amount: ");
            b.deposit(sc.nextDouble());
```

```
        } else if (ch == 2) {
            System.out.print("Enter Amount: ");
            b.withdraw(sc.nextDouble());
        }
  b.showBalance();
  sc.close();
    }
}
```

OUTPUT:

```
Enter Initial Balance: 5000
1.Deposit
2.Withdraw
3.Show Balance
Enter Choice: 2
Enter Amount: 2000
Balance = 3000.0
```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

PROGRAM:

```
import java.util.Scanner;

class Admission {

void calculateFee() {

        System.out.println("Undergraduate Fee = 50000");

    }

void calculateFee(int pg) {

        System.out.println("Postgraduate Fee = 70000");

    }

 void calculateFee(double scholarship) {

        System.out.println("Scholarship Student Fee = 30000");

    }

}

public class UniversityAdmission {

    public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

        Admission a = new Admission();

System.out.println("1.Undergraduate");

        System.out.println("2.Postgraduate");

        System.out.println("3.Scholarship");

        System.out.print("Enter Choice: ");

        int ch = sc.nextInt();

if (ch == 1)
```

```
            a.calculateFee();
        else if (ch == 2)
            a.calculateFee(1);
        else if (ch == 3)
            a.calculateFee(1.0);
        else
            System.out.println("Invalid Choice");
 sc.close();
   }
}
```

OUTPUT:

```
1.Undergraduate
2.Postgraduate
3.Scholarship
Enter Choice: 1
Undergraduate Fee = 50000
```

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

PROGRAM:

```
import java.util.Scanner;
abstract class Shape {
    abstract void area();
```

```
}
class Circle extends Shape {
    double r;
    Circle(double r) {
        this.r = r;
    }
    void area() {
       System.out.println("Circle Area = " + (3.14 * r * r));
    }
}
class Rectangle extends Shape {
    double l, b;

    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }
    void area() {
        System.out.println("Rectangle Area = " + (l * b));
    }
}
class Triangle extends Shape {
    double b, h;
```

```
    Triangle(double b, double h) {
        this.b = b;
        this.h = h;
    }
 void area() {
        System.out.println("Triangle Area = " + (0.5 * b * h));
    }
}
public class ShapeCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Shape[] s = new Shape[3];
  s[0] = new Circle(5);
        s[1] = new Rectangle(4, 6);
        s[2] = new Triangle(3, 8);
for (int i = 0; i < 3; i++) {
            s[i].area();   // Runtime Polymorphism
        }
 sc.close();
    }
}
```

OUTPUT:

```
Circle Area = 78.5
```

```
Rectangle Area = 24.0

Triangle Area = 12.0
```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

PROGRAM:

```
 import java.util.Scanner;

interface MedicalRecord {

    void addRecord();

    void displayRecord();

}

class Patient implements MedicalRecord {

    String name;

    int age;

    public void addRecord() {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Patient Name: ");

        name = sc.nextLine();

        System.out.print("Enter Age: ");

        age = sc.nextInt();

        System.out.println("Patient Record Added");

    }

    public void displayRecord() {
```

```
        System.out.println("Patient Name: " + name);
        System.out.println("Age: " + age);
    }
}
class Doctor implements MedicalRecord {
    String name;
    String department;
    public void addRecord() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Doctor Name: ");
        name = sc.nextLine();
        System.out.print("Enter Department: ");
        department = sc.nextLine();
        System.out.println("Doctor Record Added");
    }
    public void displayRecord() {
        System.out.println("Doctor Name: " + name);
        System.out.println("Department: " + department);
    }
}
public class HospitalManagement {
    public static void main(String[] args) {
        MedicalRecord m;
```

```
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Patient");
        System.out.println("2. Doctor");
        System.out.print("Enter Choice: ");
        int ch = sc.nextInt();
        sc.nextLine();
        if (ch == 1)
            m = new Patient();
        else
            m = new Doctor();
        m.addRecord();
        m.displayRecord();
        sc.close();
    }
}
```

OUTPUT:

```
1. Patient
2. Doctor
Enter Choice: 1
Enter Patient Name: AKASH
Enter Age: 18
Patient Record Added
Patient Name: AKASH
Age: 18
```

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

PROGRAM:

```
 class Product {

    String name;

    int price;

    Product(String name, int price) {

        this.name = name;

        this.price = price;

    }

}

class Order {

    void calculateBill(Product p, int qty) {

        int total = p.price * qty;

        System.out.println("Product Name: " + p.name);

        System.out.println("Quantity: " + qty);

        System.out.println("Total Amount: " + total);

    }

}

public class ShoppingApp {
```

```
    public static void main(String[] args) {

        Product p = new Product("Mobile", 20000);

        Order o = new Order();

        o.calculateBill(p, 2);

    }

}
```

OUTPUT:

```
Product Name: Mobile

Quantity: 2

Total Amount: 40000
```

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

PROGRAM:

```
 import java.util.Scanner;

class Student {

    int roll;

    String name;

    int m1, m2, m3;

    Student(int roll, String name, int m1, int m2, int m3) {

        this.roll = roll;

        this.name = name;

        this.m1 = m1;
```

```
        this.m2 = m2;
        this.m3 = m3;
    }
    int total() {
        return m1 + m2 + m3;
    }
    double average() {
        return total() / 3.0;
    }
    String grade() {
        double avg = average();
        if (avg >= 90)
            return "A";
        else if (avg >= 75)
            return "B";
        else if (avg >= 50)
            return "C";
        else
            return "Fail";
    }
    void display() {
        System.out.println("Roll No: " + roll);
        System.out.println("Name: " + name);
```

```
        System.out.println("Total: " + total());
        System.out.println("Average: " + average());
        System.out.println("Grade: " + grade());
        System.out.println();
    }
}
public class StudentResult {
    public static void main(String[] args) {
        Student s[] = new Student[3];
        s[0] = new Student(1, "Rahul", 90, 85, 95);
        s[1] = new Student(2, "Priya", 70, 75, 80);
        s[2] = new Student(3, "Arun", 60, 55, 65);
        System.out.println("Student Details:");
        for (int i = 0; i < 3; i++) {
            s[i].display();
        }
        // Finding Topper
        Student topper = s[0];
        for (int i = 1; i < 3; i++) {
            if (s[i].total() > topper.total())
                topper = s[i];
        }
        System.out.println("Class Topper:");
```

```
        System.out.println(topper.name);
        System.out.println("Total Marks: " + topper.total());
    }
}
```

OUTPUT:

```
Student Details:
Roll No: 1
Name: Rahul
Total: 270
Average: 90.0
Grade: A

Roll No: 2
Name: Priya
Total: 225
Average: 75.0
Grade: B

Roll No: 3
Name: Arun
Total: 180
Average: 60.0
Grade: C

Class Topper:
Rahul
```

Total Marks: 270

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

PROGRAM:

```
 class Device {
    void turnOn() {
        System.out.println("Device is ON");
    }
    void turnOff() {
        System.out.println("Device is OFF");
    }
    void power() {
        System.out.println("Power Consumption");
    }
}
class Light extends Device {
    void power() {
        System.out.println("Light Power = 60W");
    }
}
class Fan extends Device {
    void power() {
        System.out.println("Fan Power = 75W");
    }
}
class AirConditioner extends Device {
    void power() {
        System.out.println("AC Power = 1500W");
    }
}
public class SmartHome {
    public static void main(String[] args) {
        Device d;
```

```
        d = new Light();
        d.turnOn();
        d.power();
        d.turnOff();
        System.out.println();
        d = new Fan();
        d.turnOn();
        d.power();
        d.turnOff();
        System.out.println();
        d = new AirConditioner();
        d.turnOn();
        d.power();
        d.turnOff();
    }
}
```

OUTPUT:

```
Device is ON
Light Power = 60W
Device is OFF

Device is ON
Fan Power = 75W
Device is OFF

Device is ON
AC Power = 1500W
Device is OFF
```

Rubrics for Calculation

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks	
Problem Understan ding	20%	Clearly understands problem	Understands problem with minor gaps in constraints	Partial understandi ng; misses	Misinterpret s problem or constraints		

		constraints, edge cases, and competitive requirements		key constraints			
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach		
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results		
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints		
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases		

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____